

JavaScript + TypeScript SDET Interview Guide

70 Senior SDET / Test Automation Lead Questions — Answers + Practical Examples

Prepared from the uploaded JavaScript + TypeScript SDET Interview Guide, expanded so that every question includes a practical example. The original guide covers JavaScript fundamentals, ES6+, functions, scope, closures, objects, arrays, asynchronous JavaScript, Promises, error handling, TypeScript and Playwright framework concepts. [filecite turn0file0 L3-L8](#)

Interview focus: For a 13+ years SDET profile, use the examples to explain not only syntax but also where the concept is useful in an automation framework.

1. JavaScript Fundamentals

Q1. What is JavaScript?

Interview-ready answer: JavaScript is a dynamically typed programming language used for web and server-side development. It supports procedural, object-oriented and functional programming.

Practical example:

```
let browser = "Chrome";
function greet(name) {
    return "Hello " + name;
}
console.log(greet("Swapnil"));
```

Q2. Difference between var, let and const?

Interview-ready answer: var is function-scoped and can be redeclared. let and const are block-scoped. let can be reassigned; const cannot be reassigned.

Practical example:

```
var x = 10;
var x = 20;           // allowed

let y = 10;
y = 20;               // allowed

const z = 10;
// z = 20;           // TypeError

if (true) {
    let blockValue = 100;
}
// console.log(blockValue); // ReferenceError
```

Q3. What is hoisting?

Interview-ready answer: JavaScript processes declarations before executing code. var is initialised as undefined; let and const are hoisted but remain in the Temporal Dead Zone until declaration.

Practical example:

```
console.log(a); // undefined
var a = 10;

showMessage(); // works
function showMessage() {
    console.log("Hello");
}
```

Q4. What is the Temporal Dead Zone?

Interview-ready answer: The TDZ is the period between entering a scope and the declaration of a let/const variable. Accessing it during this period throws a ReferenceError.

Practical example:

```
// console.log(name); // ReferenceError
let name = "Swapnil";
console.log(name);
```

Q5. Primitive vs non-primitive data types?

Interview-ready answer: Primitive types include string, number, boolean, undefined, null, bigint and symbol. Objects, arrays and functions are non-primitive reference types.

Practical example:

```
let age = 35;           // primitive
let name = "Swapnil";   // primitive
let user = { name: "Swapnil" }; // object
let numbers = [1, 2, 3]; // array

user.name = "Test Lead"; // object can be modified
```

Q6. == vs ===?

Interview-ready answer: == performs type coercion before comparison. === compares both value and type and is generally preferred in automation code.

Practical example:

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false

const statusCode = "200";
console.log(statusCode === 200); // false
```

Q7. What is type coercion?

Interview-ready answer: Type coercion is automatic conversion of one data type into another during an operation.

Practical example:

```
console.log("5" - 2); // 3
console.log("5" + 2); // "52"

let responseCode = "200";
if (Number(responseCode) === 200) {
  console.log("Success");
}
```

Q8. null vs undefined?

Interview-ready answer: undefined generally means a value has not been assigned. null is an intentional representation of no value.

Practical example:

```
let token;
console.log(token); // undefined

let apiResponse = null;
console.log(apiResponse); // intentionally no response
```

Q9. What is NaN?

Interview-ready answer: NaN means Not-a-Number and is produced by invalid numeric operations. Number.isNaN() is a reliable check.

Practical example:

```
const result = Number("abc");
console.log(result); // NaN

console.log(Number.isNaN(result)); // true
```

Q10. typeof vs instanceof?

Interview-ready answer: typeof identifies primitive/basic types. instanceof checks whether an object is associated with a particular constructor or class.

Practical example:

```
const numbers = [1, 2, 3];

console.log(typeof 10); // "number"
console.log(typeof "hello"); // "string"
console.log(numbers instanceof Array); // true
```

2. Functions, Scope & Closures

Q11. Function declaration vs expression?

Interview-ready answer: A declaration uses function name(...) {} and is hoisted differently from a function expression assigned to a variable.

Practical example:

```
// Declaration: can be called before its definition
sayHello();
function sayHello() {
  console.log("Hello");
}

// Expression: variable must be initialised first
const add = function(a, b) {
  return a + b;
};
console.log(add(2, 3));
```

Q12. What is an arrow function?

Interview-ready answer: Arrow functions provide concise syntax and do not have their own this.

Practical example:

```
const add = (a, b) => a + b;
console.log(add(10, 20));

// Useful for array operations
const values = [1, 2, 3];
const doubled = values.map(value => value * 2);
console.log(doubled);
```

Q13. What is a callback?

Interview-ready answer: A callback is a function passed to another function to be invoked later or as part of an operation.

Practical example:

```
function executeTest(testName, callback) {
  console.log("Running: " + testName);
  callback();
}

executeTest("Login test", function() {
  console.log("Test completed");
});
```

Q14. What is a higher-order function?

Interview-ready answer: A higher-order function accepts another function as an argument or returns a function.

Practical example:

```
function calculate(a, b, operation) {
  return operation(a, b);
}

const result = calculate(10, 5, (x, y) => x - y);
console.log(result); // 5
```

Q15. What is scope?

Interview-ready answer: Scope determines where variables can be accessed. Common forms are global, function and block scope.

Practical example:

```
let globalValue = "global";

function test() {
  let functionValue = "function";

  if (true) {
    let blockValue = "block";
    console.log(globalValue, functionValue, blockValue);
  }
}

test();
```

Q16. What is lexical scope?

Interview-ready answer: A function can access variables based on where it was defined, rather than where it is called.

Practical example:

```
const environment = "QA";

function test() {
  console.log(environment);
}

function run() {
  test();
}

run(); // QA
```

Q17. What is a closure?

Interview-ready answer: A closure is created when an inner function retains access to variables from its outer function even after the outer function has finished.

Practical example:

```
function createCounter() {
  let count = 0;

  return function() {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

3. Objects & Modern JavaScript

Q18. How do you create an object?

Interview-ready answer: An object stores related data as key-value pairs and can also contain methods.

Practical example:

```
const user = {
  name: "Swapnil",
  role: "SDET",
  experience: 13
};

console.log(user.name);
console.log(user.role);
```

Q19. What is destructuring?

Interview-ready answer: Destructuring extracts values from arrays or properties from objects into variables.

Practical example:

```
const user = {
  name: "Swapnil",
  role: "SDET"
};

const { name, role } = user;
console.log(name);
console.log(role);

const numbers = [10, 20];
const [first, second] = numbers;
```

Q20. What is the spread operator?

Interview-ready answer: The ... operator expands iterable values or object properties. It is commonly used to copy or merge arrays and objects.

Practical example:

```
const baseConfig = {
  browser: "chromium",
  headless: true
};

const testConfig = {
  ...baseConfig,
  timeout: 30000
};

console.log(testConfig);
```

Q21. What is the rest operator?

Interview-ready answer: The ... parameter collects multiple function arguments into an array.

Practical example:

```
function sum(...numbers) {
  return numbers.reduce((total, value) => total + value, 0);
}

console.log(sum(10, 20, 30)); // 60
```

Q22. Shallow copy vs deep copy?

Interview-ready answer: A shallow copy copies top-level properties while nested objects may remain shared references. A deep copy recursively copies nested data; `structuredClone()` is a modern option for many data types.

Practical example:

```
const original = {
  name: "Swapnil",
  address: { city: "Pune" }
};

const shallow = { ...original };
shallow.address.city = "Mumbai";

console.log(original.address.city); // Mumbai

const deep = structuredClone(original);
deep.address.city = "Delhi";
console.log(original.address.city);
```

4. Arrays

Q23. map vs filter vs reduce?

Interview-ready answer: map transforms each element and returns a new array. filter returns matching elements. reduce combines elements into one accumulated result.

Practical example:

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(n => n * 2);
const even = numbers.filter(n => n % 2 === 0);
const total = numbers.reduce((sum, n) => sum + n, 0);

console.log(doubled); // [2,4,6,8]
console.log(even);    // [2,4]
console.log(total);   // 10
```

Q24. forEach vs map?

Interview-ready answer: forEach is for iteration and does not create a transformed result array. map returns a new array.

Practical example:

```
const numbers = [1, 2, 3];

numbers.forEach(n => console.log(n));

const result = numbers.map(n => n * 10);
console.log(result); // [10,20,30]
```

Q25. How do you remove duplicates?

Interview-ready answer: Use Set to keep only unique values.

Practical example:

```
const values = [10, 20, 20, 30, 10];
const unique = [...new Set(values)];

console.log(unique); // [10,20,30]
```

Q26. How do you find array frequency?

Interview-ready answer: Use an object or Map as a frequency counter and increment the count for each value.

Practical example:

```
const values = ["PASS", "FAIL", "PASS", "PASS"];

const frequency = {};
values.forEach(value => {
  frequency[value] = (frequency[value] || 0) + 1;
});

console.log(frequency); // { PASS: 3, FAIL: 1 }
```

5. Asynchronous JavaScript

Q27. Synchronous vs asynchronous execution?

Interview-ready answer: Synchronous statements execute in sequence. Asynchronous operations can complete later, allowing other work to proceed.

Practical example:

```
console.log("Start");

setTimeout(() => {
  console.log("Async operation completed");
}, 1000);

console.log("End");
// Start
// End
// Async operation completed
```

Q28. What is a Promise?

Interview-ready answer: A Promise represents the eventual success or failure of an asynchronous operation. Its states are pending, fulfilled and rejected.

Practical example:

```
const login = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Login successful");
  } else {
    reject(new Error("Login failed"));
  }
});

login.then(result => console.log(result))
  .catch(error => console.error(error.message));
```

Q29. What are async and await?

Interview-ready answer: async functions return Promises. await pauses execution within an async function until a Promise settles, making asynchronous code easier to read.

Practical example:

```
async function getUser() {
  const response = await fetch("https://example.com/user");
  const data = await response.json();
  console.log(data);
}

getUser();
```

Q30. Promise.all vs Promise.allSettled?

Interview-ready answer: Promise.all rejects when one Promise rejects. Promise.allSettled waits for all Promises and reports each outcome.

Practical example:

```
const tests = [
  Promise.resolve("Login PASS"),
  Promise.resolve("Search PASS"),
  Promise.reject("Payment FAIL")
];

Promise.all(tests)
  .then(results => console.log(results))
  .catch(error => console.log("One failed:", error));

Promise.allSettled(tests)
  .then(results => console.log(results));
```

Q31. What is the event loop?

Interview-ready answer: The event loop coordinates the call stack with asynchronous queues. Promise callbacks are microtasks and generally run before timer callbacks after the current stack completes.

Practical example:

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

console.log("4");

// Output:
// 1
// 4
// 3
// 2
```

Q32. Explain this output.

Interview-ready answer: The synchronous statements run first. The Promise callback is a microtask, so it runs before the timer callback.

Practical example:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// Output: 1, 4, 3, 2
```

6. Error Handling

Q33. How do you handle exceptions?

Interview-ready answer: Use try/catch/finally. try contains risky code, catch handles errors, and finally runs regardless of success or failure.

Practical example:

```
try {
  const data = JSON.parse("invalid-json");
  console.log(data);
} catch (error) {
  console.log("Parsing failed:", error.message);
} finally {
  console.log("Cleanup completed");
}
```

Q34. How do you throw a custom error?

Interview-ready answer: Use throw new Error('message'). Custom Error classes can also be created by extending Error.

Practical example:

```
class LoginError extends Error {
  constructor(message) {
    super(message);
    this.name = "LoginError";
  }
}

function login(success) {
  if (!success) {
    throw new LoginError("Invalid credentials");
  }
}

try {
  login(false);
} catch (error) {
  console.log(error.name, error.message);
}
```

7. JavaScript Programming Questions

Q35. Reverse a string.

Interview-ready answer: Use split/reverse/join or implement the logic manually.

Practical example:

```
function reverseString(str) {
  return str.split("").reverse().join("");
}

console.log(reverseString("JavaScript"));
// tpircSavaJ
```

Q36. Reverse a string without reverse().

Interview-ready answer: Loop from the last character to the first and append each character.

Practical example:

```
function reverseString(str) {
  let result = "";

  for (let i = str.length - 1; i >= 0; i--) {
    result += str[i];
  }

  return result;
}

console.log(reverseString("SDET")); // TEDS
```

Q37. Check palindrome.

Interview-ready answer: A palindrome reads the same forward and backward.

Practical example:

```
function isPalindrome(str) {
  const reversed = str.split("").reverse().join("");
  return str === reversed;
}

console.log(isPalindrome("madam")); // true
console.log(isPalindrome("hello")); // false
```

Q38. Count character occurrences.

Interview-ready answer: Create a frequency object and increment the count for every character.

Practical example:

```
function countCharacters(str) {
  const count = {};

  for (let char of str) {
    count[char] = (count[char] || 0) + 1;
  }

  return count;
}

console.log(countCharacters("selenium"));
// { s: 1, e: 2, l: 1, n: 1, i: 1, u: 1, m: 1 }
```

Q39. Find duplicate characters.

Interview-ready answer: Build a frequency map and print characters whose count is greater than 1.

Practical example:

```
function duplicateCharacters(str) {
  const count = {};

  for (let char of str) {
    count[char] = (count[char] || 0) + 1;
  }

  return Object.keys(count).filter(char => count[char] > 1);
}

console.log(duplicateCharacters("programming"));
// ["r", "g", "m"]
```

Q40. Find first non-repeated character.

Interview-ready answer: Build a frequency map, then scan the original string and return the first character with count 1.

Practical example:

```
function firstNonRepeated(str) {
  const count = {};

  for (let char of str) {
    count[char] = (count[char] || 0) + 1;
  }

  for (let char of str) {
    if (count[char] === 1) return char;
  }

  return null;
}

console.log(firstNonRepeated("swiss")); // w
```

Q41. Find largest number.

Interview-ready answer: Initialise max with the first element and scan the array, updating max when a larger value is found.

Practical example:

```
function largest(arr) {
  let max = arr[0];

  for (let num of arr) {
    if (num > max) {
      max = num;
    }
  }

  return max;
}

console.log(largest([10, 50, 20, 90, 30])); // 90
```

Q42. Find second largest number.

Interview-ready answer: Track largest and second-largest values in one pass.

Practical example:

```
function secondLargest(arr) {
  let largest = -Infinity;
  let second = -Infinity;

  for (let num of arr) {
    if (num > largest) {
      second = largest;
      largest = num;
    } else if (num > second && num !== largest) {
      second = num;
    }
  }

  return second;
}

console.log(secondLargest([10, 50, 20, 90, 30])); // 50
```

Q43. Find duplicate numbers.

Interview-ready answer: Maintain a Set of seen values and another Set for values encountered more than once.

Practical example:

```
function findDuplicates(arr) {
  const seen = new Set();
  const duplicates = new Set();

  for (let value of arr) {
    if (seen.has(value)) {
      duplicates.add(value);
    } else {
      seen.add(value);
    }
  }

  return [...duplicates];
}

console.log(findDuplicates([1,2,3,2,4,3]));
// [2,3]
```

Q44. Remove duplicate numbers.

Interview-ready answer: Use Set to remove repeated values.

Practical example:

```
const numbers = [1, 2, 2, 3, 3, 4];
const unique = [...new Set(numbers)];

console.log(unique); // [1,2,3,4]
```

Q45. Find missing number.

Interview-ready answer: For an array containing 1..n with one missing value, calculate $n*(n+1)/2$ and subtract the actual sum.

Practical example:

```
function missingNumber(arr, n) {
  const expected = n * (n + 1) / 2;
  const actual = arr.reduce((sum, value) => sum + value, 0);
  return expected - actual;
}

console.log(missingNumber([1,2,3,5], 5)); // 4
```

Q46. Check anagrams.

Interview-ready answer: Normalise both strings, sort their characters and compare the resulting strings.

Practical example:

```
function isAnagram(str1, str2) {
  const a = str1.toLowerCase().split('').sort().join('');
  const b = str2.toLowerCase().split('').sort().join('');
  return a === b;
}

console.log(isAnagram("listen", "silent")); // true
```

8. TypeScript Theory

Q47. What is TypeScript?

Interview-ready answer: TypeScript is a statically typed superset of JavaScript. It adds types and other language features and is transpiled to JavaScript.

Practical example:

```
let username: string = "Swapnil";
let experience: number = 13;

// username = 100; // Compile-time error
```

Q48. Why use TypeScript in automation?

Interview-ready answer: Static type checking, better IDE support, safer refactoring, interfaces, generics and maintainability are useful in large automation frameworks.

Practical example:

```
interface TestUser {
  username: string;
  role: string;
}

function createUser(user: TestUser) {
  console.log(user.username);
}

createUser({
  username: "qa_user",
  role: "SDET"
});
```

Q49. JavaScript vs TypeScript?

Interview-ready answer: JavaScript is dynamically typed. TypeScript adds a type system and is transformed into JavaScript before execution.

Practical example:

```
// JavaScript
let id = 101;
id = "101"; // allowed

// TypeScript
let userId: number = 101;
// userId = "101"; // compile-time error
```

Q50. What is an interface?

Interview-ready answer: An interface defines the expected structure of an object or class.

Practical example:

```
interface Employee {
  id: number;
  name: string;
  role: string;
}

const employee: Employee = {
  id: 101,
  name: "Swapnil",
  role: "SDET"
};
```

Q51. Interface vs type?

Interview-ready answer: Both can describe object shapes. Interfaces support declaration merging and are commonly used for object contracts; type aliases are flexible for unions and intersections.

Practical example:

```
interface User {
  name: string;
}

type Status = "PASS" | "FAIL";

const user: User = { name: "Swapnil" };
const status: Status = "PASS";
```

Q52. What is a union type?

Interview-ready answer: A union type allows more than one possibility.

Practical example:

```
let id: string | number;

id = 101;
id = "USR-101";
```

Q53. What is an intersection type?

Interview-ready answer: An intersection combines multiple types into one type.

Practical example:

```
type Employee = {
  id: number;
};

type Tester = {
  automation: boolean;
};

type SDET = Employee & Tester;

const person: SDET = {
  id: 101,
  automation: true
};
```

Q54. What are generics?

Interview-ready answer: Generics allow reusable type-safe code.

Practical example:

```
function getValue<T>(value: T): T {
  return value;
}

const numberValue = getValue<number>(100);
const stringValue = getValue<string>("SDET");
```

Q55. What is any?

Interview-ready answer: any disables most compile-time type checking for that value and should be used sparingly.

Practical example:

```
let data: any = "PASS";
data = 100;
data = { status: true };

console.log(data);
```

Q56. any vs unknown vs never?

Interview-ready answer: any permits almost anything; unknown is safer and requires type narrowing; never represents values that cannot occur or functions that never return normally.

Practical example:

```
let a: any = "hello";

let b: unknown = "hello";
if (typeof b === "string") {
  console.log(b.toUpperCase());
}

function fail(message: string): never {
  throw new Error(message);
}
```

9. TypeScript + Playwright

Q57. How do you type a Playwright Page Object?

Interview-ready answer: Use Page and Locator types. Store private Page/Locator fields and expose business-level methods such as login(username, password).

Practical example:

```
import { Page, Locator } from "@playwright/test";

export class LoginPage {
  private username: Locator;
  private password: Locator;
  private loginButton: Locator;

  constructor(private page: Page) {
    this.username = page.locator("#username");
    this.password = page.locator("#password");
    this.loginButton = page.locator("#login");
  }

  async login(username: string, password: string): Promise<void> {
    await this.username.fill(username);
    await this.password.fill(password);
    await this.loginButton.click();
  }
}
```

Q58. Why use private in a Page Object?

Interview-ready answer: It encapsulates locator and implementation details so tests interact through stable business methods.

Practical example:

```
class LoginPage {
  constructor(private page: Page) {}

  private username = this.page.locator("#username");

  async login(name: string) {
    await this.username.fill(name);
  }
}

// Test uses login(), not the locator directly.
```

Q59. How do you type an API response?

Interview-ready answer: Define an interface for the expected response structure and use it for response data.

Practical example:

```
interface UserResponse {
  id: number;
  name: string;
  email: string;
}

const user: UserResponse = {
  id: 101,
  name: "Swapnil",
  email: "qa@example.com"
};
```

Q60. How do you create a generic API response?

Interview-ready answer: Define a generic wrapper so the same response structure can work with different payload types.

Practical example:

```
interface ApiResponse<T> {
  status: number;
  message: string;
  data: T;
}

interface User {
  id: number;
  name: string;
}

const response: ApiResponse<User> = {
  status: 200,
  message: "Success",
  data: {
    id: 101,
    name: "Swapnil"
  }
};
```

10. Senior SDET Coding Practice

Q61. Implement map().

Interview-ready answer: Loop through an input array, apply a callback to each element and return a new array.

Practical example:

```
function customMap(arr, callback) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    result.push(callback(arr[i], i, arr));
  }

  return result;
}

console.log(customMap([1,2,3], n => n * 2));
// [2,4,6]
```

Q62. Implement filter().

Interview-ready answer: Loop through the array, call the predicate and push only elements for which the predicate returns true.

Practical example:

```
function customFilter(arr, predicate) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    if (predicate(arr[i], i, arr)) {
      result.push(arr[i]);
    }
  }

  return result;
}

console.log(customFilter([1,2,3,4], n => n % 2 === 0));
// [2,4]
```

Q63. Implement reduce().

Interview-ready answer: Maintain an accumulator, update it using a callback for each element and return the final accumulator.

Practical example:

```
function customReduce(arr, callback, initialValue) {
  let accumulator = initialValue;

  for (let i = 0; i < arr.length; i++) {
    accumulator = callback(accumulator, arr[i], i, arr);
  }

  return accumulator;
}

const total = customReduce([10,20,30], (sum, n) => sum + n, 0);
console.log(total); // 60
```

Q64. Implement debounce.

Interview-ready answer: Return a function that clears the previous timer and schedules the callback after the specified delay.

Practical example:

```
function debounce(callback, delay) {
  let timer;

  return function(...args) {
    clearTimeout(timer);

    timer = setTimeout(() => {
      callback(...args);
    }, delay);
  };
}

const search = debounce(value => {
  console.log("Search API:", value);
}, 500);

search("pla");
search("play");
search("playwright");
// Only the last call runs after the delay.
```

Q65. Implement throttle.

Interview-ready answer: Return a function that permits execution at most once during a defined interval.

Practical example:

```
function throttle(callback, interval) {
  let lastRun = 0;

  return function(...args) {
    const now = Date.now();

    if (now - lastRun >= interval) {
      lastRun = now;
      callback(...args);
    }
  };
}

const logScroll = throttle(() => {
  console.log("Scroll handled");
}, 1000);
```

Q66. Implement custom Promise.all().

Interview-ready answer: Accept an array of Promises, preserve result order, resolve when all succeed, and reject when the first Promise rejects.

Practical example:

```
function customPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completed = 0;

    if (promises.length === 0) {
      resolve([]);
      return;
    }

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then(value => {
          results[index] = value;
          completed++;

          if (completed === promises.length) {
            resolve(results);
          }
        })
        .catch(reject);
    });
  });
}

customPromiseAll([
  Promise.resolve("Login"),
  Promise.resolve("Search")
]).then(console.log);
```

Q67. Implement memoization.

Interview-ready answer: Cache function results using a Map keyed by input so repeated calls can return cached results.

Practical example:

```
function memoize(fn) {
  const cache = new Map();

  return function(value) {
    if (cache.has(value)) {
      console.log("From cache");
      return cache.get(value);
    }

    const result = fn(value);
    cache.set(value, result);
    return result;
  };
}

const square = memoize(n => n * n);

console.log(square(5)); // calculates
console.log(square(5)); // cached
```

Q68. Implement retry logic.

Interview-ready answer: Call an asynchronous operation, catch failures, retry up to a configured count and optionally wait between attempts.

Practical example:

```
async function retry(operation, retries, delay) {
  let lastError;

  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      if (attempt < retries) {
        await new Promise(resolve => setTimeout(resolve, delay));
      }
    }
  }

  throw lastError;
}

retry(() => fetch("https://example.com/api"), 3, 1000);
```

Q69. Implement polling.

Interview-ready answer: Repeatedly check a condition at an interval until it succeeds or a timeout is reached.

Practical example:

```
async function poll(check, interval, timeout) {
  const start = Date.now();

  while (Date.now() - start < timeout) {
    if (await check()) {
      return true;
    }

    await new Promise(resolve => setTimeout(resolve, interval));
  }

  throw new Error("Polling timed out");
}

// Example: wait until an order becomes READY
poll(async () => {
  const status = await getOrderStatus();
  return status === "READY";
}, 2000, 30000);
```

Q70. Create a typed Playwright API client.

Interview-ready answer: Create reusable methods with typed request payloads and response interfaces, centralising authentication, request handling and common assertions.

Practical example:

```
import { APIRequestContext } from "@playwright/test";

interface User {
  id: number;
  name: string;
}

interface CreateUserRequest {
  name: string;
}

export class UserApi {
  constructor(private request: APIRequestContext) {}

  async createUser(payload: CreateUserRequest): Promise<User> {
    const response = await this.request.post("/users", {
      data: payload
    });

    if (!response.ok()) {
      throw new Error("Create user failed");
    }

    return await response.json() as Promise<User>;
  }
}

// Test usage:
// const user = await userApi.createUser({ name: "Swapnil" });
// console.log(user.id);
```

Senior SDET Interview Priority

The uploaded guide highlights these areas as the main priorities for senior SDET interviews: `var/let/const`, hoisting, TDZ, equality, scope, closure and `this`; `map/filter/reduce`, `Set/Map` and destructuring/spread/rest; `Promise`, `async/await`, event loop and `Promise.all/allSettled`; string and array coding; TypeScript interfaces/types/unions/intersections/generics; and Playwright Page Objects, fixtures, API typing and reusable utilities. ■filecite■turn0file0■L303-L310■

Senior framework coding: `retry`, `polling`, `debounce`, `throttle` and `memoization` are also explicitly listed in the source guide. ■filecite■turn0file0■L313-L315■

How to use this PDF in an interview

1. Give the definition in 20–30 seconds.
2. Explain the example line-by-line if the interviewer asks.
3. Connect the concept to automation: test data, API clients, Playwright Page Objects, `waits`, `retries`, fixtures or reusable utilities.
4. For coding questions, first explain the approach and complexity, then code.